

Espressioni: no categoria sintattica

- senza identificatori
- con identificatori
 - associano un nome a valori denotabili

↓
rapp. valori esprimibili

Semantica (dinamica) delle espressioni: no descrive formalmente il processo di valutazione delle espressioni

↓
parte dell'assunzione che l'espressione sia scritta "correttamente"

~~2+true~~

Tipo no caratterizza l'insieme dei valori denotabili da un id e l'insieme degli operatori applicabili

Definizione di un sistema di regole che permettono di verificare la corretta costruzione (in termini di tipo) delle espressioni

SEMANTICA STATICA

Ambiente statico: insieme di legami tra identificatori e tipi

$D_{Typ} = \{int, bool\}$
tipi denotabili

$[D_{val}] = N \cup B$

$T_{env} = \bigcup_{v \in Id} T_{env_I}$
soluzione finita

$T_{env_I} : I \rightarrow D_{Typ}$ ad ogni identificatore in I associa un tipo in D_{Typ}

Metavariable: $\Delta \in T_{env}$ generico ambiente statico

$\Delta \vdash_I e : \tau$ asserisce che e è di tipo $\tau \in D_{Typ}$ nell'ambiente statico Δ definito sugli identificatori in I

dominio di Δ

Forma delle regole

te: $\mathbb{Z}$

[0:15:00]

Se le regole della semantica statica permettono di associare un tipo ad una espressione allora diciamo che l'espressione è WELL FORMED.

$$E ::= A \mid B$$

$$A ::= x \mid n \mid A \text{ op } A \quad \text{op} \in \{+, -, *\}$$

$$B ::= x \mid \text{true} \mid \text{false} \mid \text{not } B \mid B \text{ or } B \mid A = A$$

semantica statica

$$E_{s1}: \vdash n : \text{int}$$

W/V è possibile applicare la regola

$$E_{s2}: \vdash t : \text{bool}$$

$$E_{s3}: \Delta \vdash_I x : \mathbb{Z} \quad \text{se } x \in I, \Delta(x) = \mathbb{Z}$$

$$E_{s4}: \frac{\Delta \vdash_I e_1 : \text{int} \quad \Delta \vdash_I e_2 : \text{int} \rightarrow \text{promessa da verificare}}{\Delta \vdash_I e_1 \text{ op } e_2 : \text{int}}$$

Base risultato

$$E_{s5}: \frac{\Delta \vdash_I e_1 : \text{bool} \quad \Delta \vdash_I e_2 : \text{bool}}{\Delta \vdash_I e_1 \text{ or } e_2 : \text{bool}}$$

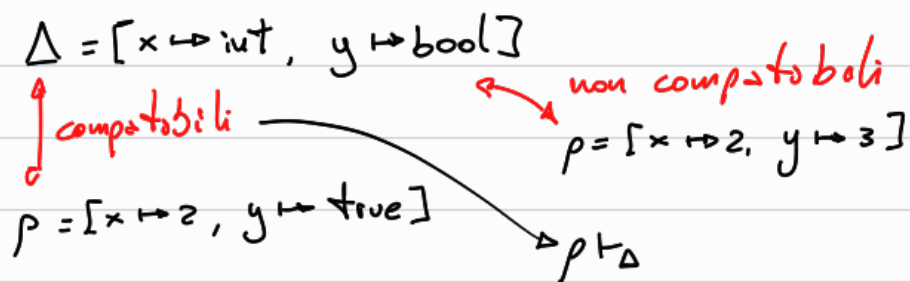
$$E_{s6}: \frac{\Delta \vdash_I e : \text{bool}}{\Delta \vdash_I \text{not } e : \text{bool}}$$

$$E_{s7}: \frac{\Delta \vdash_I e_1 : \text{int} \quad \Delta \vdash_I e_2 : \text{int}}{\Delta \vdash_I e_1 = e_2 : \text{bool}}$$

Ambient: compatibili: $\rho \in \text{Env}_I$ (ambiente dinamico che associa valori agli identificatori in I)

$\Delta \in \text{TEnv}_I$ (ambiente statico che associa tipi agli identificatori in I)

ρ e Δ sono compatibili se $\forall id \in I. \Delta(id) = \mathbb{Z}$ sse $\rho(id) \in \mathbb{Z}$
(è un valore di tipo $\mathbb{Z}$)



=> Per definire gli ambienti (sia Δ che ρ) abbiamo bisogno di una categoria sintattica che permetta di creare e modificare legami che vanno a costruire gli ambienti:

DICHIARAZIONI: categoria sintattica che fornisce il meccanismo (implicito o esplicito) per creare legami:

denotano richieste di modifica/creazione di ambienti
 tra identificatori e tipi (statico) o oggetti (dinamico)

Le dichiarazioni vengono **ELABORATE** per attuare le richieste di modifica/creazione di ambienti

SINTASSI

$D : D \rightarrow \text{id} \mid \text{const } x : \tau = e \mid D ; D \mid D \text{ in } D \mid \rho$

corrisponde $\Delta = \emptyset$
 dichiarazione un id costante di nome x tipo τ e valore rappresentato da e
 composizione sequenziale
 composizione privata
 ambiente dinamico (presenza nella grammatica per motivi formali)

[0:45:56]

Aggiornamento di ambiente

ρ, ρ' ambienti
 $\rho[\rho']$ ρ modificato da ρ'
 $\rho : I \quad I \subseteq_f Id \quad (\rho \text{ definito su } I)$
 $\rho' : I' \quad I' \subseteq_f Id \quad (\rho' \text{ definito su } I')$

$\rho[\rho'] = \bar{\rho}$ nuovo ambiente $\bar{\rho} : I \cup I'$

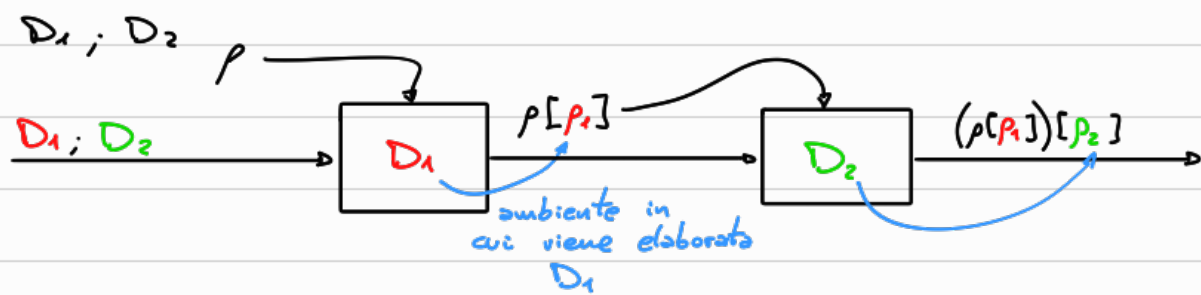
$\forall x \in I'. \bar{\beta}(x) = \beta'(x)$ $\bar{\beta}$ è un nuovo ambiente in cui prendo
 $\forall x \in I \setminus I'. \bar{\beta}(x) = \beta(x)$ tutti i legami di β e β' e per quelli
 in comune tengo solo β'

Esempio

$\rho = [x \mapsto 2, y \mapsto \text{true}, z \mapsto 5]$
 $\rho' = [x \mapsto 5, w \mapsto \text{false}]$

$\rho[\rho'] = [x \mapsto 5, w \mapsto \text{false}, y \mapsto \text{true}, z \mapsto 5]$
 sovrascriviamo $x \mapsto 2$ con $x \mapsto 5$

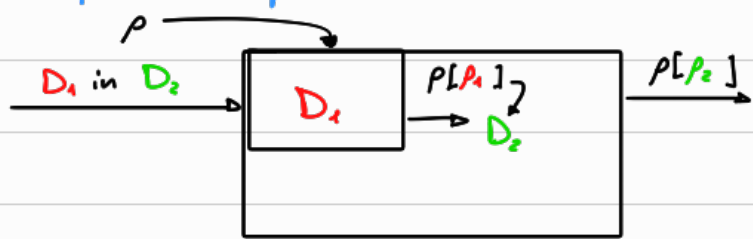
COMPOSIZIONE SEQUENZIALE



in ogni dichiarazione successivo o aggiunge nuovi legami o sovrascrive
 legami esistenti.

L'ambiente risultante è così visibile al codice successivo

Composizione privata



I legami creati o sovrascritti da D_1 sono ad
 uso esclusivo dell'elaborazione di D_2 e non visibili
 al codice successivo

Esempi

1) $\text{const } x : \text{int} = 3;$ $\rho[x \mapsto 3] = \rho$ ma usiamo per elaborare la dir. succ.
 $\text{const } x : \text{int} = 5$ in $y : \text{int} = 6 * x;$ $\rho[x \mapsto 5] = \rho_1$
 $\text{const } z : \text{int} = x + y;$ $\rho_1[y \mapsto 6 * 5 = 30] = \rho_2$
 $\rho[\rho_2] = [x \mapsto 3, y \mapsto 30, z \mapsto 33]$

2) $\text{const } x : \text{int} = 3;$ $\rho[x \mapsto 3] = \rho$
 $\text{const } x : \text{int} = 5;$ $\rho[x \mapsto 5] = \rho_1$
 $\text{const } y : \text{int} = 6 * 5;$ $\rho_1[y \mapsto 30] = \rho_2$
 $\text{const } z : \text{int} = x + y;$ $\rho_2[z \mapsto 35]$
 $\rho(\rho_2) = [x \mapsto 5, y \mapsto 30]$

Caratterizziamo identificatori in posizione libera (FI) e in posizione di definizione (DI)

$$FI(Exp \cup Dec) = I \subseteq Id \quad FI : Exp \cup Dec \rightarrow \mathcal{P}(Id)$$

associa ad ogni costrutto sintattico
l'insieme degli identificatori liberi

↓
identificatori che non hanno
un legame $\Rightarrow$ Richiedono un
contesto per avere significato

$$FI(k) = \emptyset \quad k \in \mathcal{N} \cup \mathcal{B}$$

$$FI(x) = \{x\}$$

$$FI(e_1 \text{ op } e_2) = FI(e_1) \cup FI(e_2) = FI(e_1 \text{ or } e_2)$$

$$FI(\text{not } e) = FI(e)$$

$$FI(\text{nil}) = \emptyset$$

$$FI(\text{const } x : z = e) = FI(e) \quad \text{id definito da } d_1$$

$$FI(d_1 \text{ in } d_2) = FI(d_1) \cup (FI(d_2) \setminus \underline{DI(d_1)}) = FI(d_1 \text{ in } d_2)$$

$$FI(\rho) = \emptyset$$

$DI : Exp \cup Dec \rightarrow \mathcal{P}(Id)$ Restituisce gli identificatori che nel costrutto
vengono legati a un significato che è visibile
all'esterno del costrutto

$$\forall e. DI(e) = \emptyset$$

$$DI(\text{nil}) = \emptyset$$

$$DI(\text{const } x : z = e) = \{x\}$$

$$DI(\rho) = I \quad \rho : I \equiv \rho \in Env_I$$

$$DI(d_1 \text{ in } d_2) = DI(d_2) \quad \text{solo gli identificatori legati in } d_2 \text{ hanno significato dopo l'elaborazione della composizione}$$

$DI(d_1)$ viene usato solo all'interno di d_2 quindi non ci sono

$$DI(d_1 ; d_2) = DI(d_1) \cup DI(d_2) \quad \text{tutti gli identificatori legati in } d_1 \text{ o } d_2 \text{ restano poi legati al loro significato}$$

Semantica statica delle espressioni: associa un tipo alle espressioni ben formate

=> verifica solo vincoli di tipo tra operandi

Semantica statica delle dichiarazioni: associa un ambiente statico alle dichiarazioni ben formate

$\hookrightarrow \Delta \vdash d : \Delta'$
Forma delle regole
ambiente associato e (quindi associato) da d
contesto

[1:25:00]

$DS_1: \vdash nil : \emptyset$

$DS_2: \vdash p : \Delta$ dove $p \in \Delta$ sono compatibili

$p = [x \mapsto 3, y \mapsto true]$

Le $\Delta = [x \mapsto int, y \mapsto bool]$

$DS_3: \frac{\Delta \vdash e : \tau}{\Delta \vdash const\ x : \tau = e : [x \mapsto \tau]}$

devono essere tutti uguali

$DS_4: \frac{\Delta \vdash d_1 : \Delta_1 \quad \Delta[\Delta_1] \vdash d_2 : \Delta_2}{\Delta \vdash d_1\ in\ d_2 : \Delta_2}$

$DS_5: \frac{\Delta \vdash d_1 : \Delta_1 \quad \Delta[\Delta_1] \vdash d_2 : \Delta_2}{\Delta \vdash d_1; d_2 : \Delta_1[\Delta_2]}$

entrambi visibili all'esterno